CLOUD COMPUTING / RESEARCH PROJECT

Fragmentation-Aware Scheduling for Multi-Tenant MIG-Based GPU Clouds

Auditing Minimum Fragmentation Increment (MFI) and evaluating bounded lookahead under synthetic workloads

Saurabh Kumar

IIT (BHU), Varanasi | October 2026

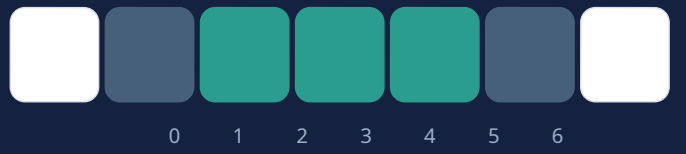
SIMULATION-BASED STUDY

Published MFI is the foundation; this project audits assumptions and evaluates whether lookahead is worth its cost.

A100 profile abstraction

Discrete placement positions

COMPUTE · 7 POSITIONS



MEMORY · 8 POSITIONS



A legal profile must satisfy BOTH footprints and the permitted start-index set.

No physical GPU benchmark or cloud deployment is claimed.

Project in One Minute

01

MIG constraints

Compute + memory + legal index positions

02

MFI baseline

Minimize immediate fragmentation increase

03

Bounded lookahead

Simulate limited future admissions

04

Paired experiments

Acceptance, uncertainty and decision cost

Current conclusion

Score semantics materially change placement choices. The current small study has not established a consistent, reliable acceptance gain from lookahead.

Contribution here = implementation audit + controlled evaluation, not a claim that lookahead is already superior.

[1]

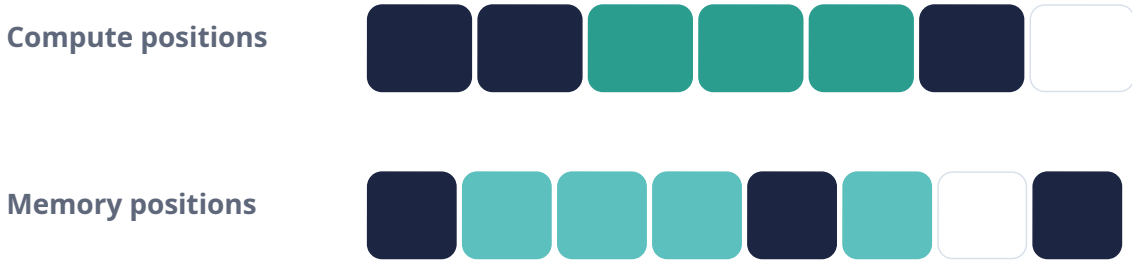
PROBLEM

Why GPU Scheduling Is Difficult in Multi-Tenant Clouds

Heterogeneous requests

- **3g.40gb** large profile
- **1g.20gb** small memory-heavy
- **2g.20gb** medium profile
- **1g.10gb** small profile

One GPU, discrete placement options



- MIG uses isolated GPU Instances with dedicated compute and memory resources.
- Profiles are discrete; only supported size combinations and legal starts are allowed.
- A placement can be feasible now yet reduce how many later jobs can be admitted.

HARDWARE-LEVEL ISOLATION · DISCRETE PROFILES

[1, Section III; 2]

MIG Profile Model Used by the Simulator

Profile	Compute	Memory	Legal starts
7g.80gb	7	8	0
4g.40gb	4	4	0
3g.40gb	3	4	0, 4
2g.20gb	2	2	0, 2, 4
1g.20gb	1	2	0, 2, 4, 6
1g.10gb	1	1	0, 1, 2, 3, 4, 5, 6

Key example: 1g.20gb = 1 compute + 2 memory slices.

A profile has TWO independent footprints

Example: 3g.40gb



3 compute slices ≠ 3 memory slices

Model boundary

A100-80GB table from the paper; not a live driver query or validated hardware emulation.

[1, Table I; 2]

CORE CONSTRAINT

Fragmentation: Enough Free Slices, but No Legal Placement

Requested profile: 3g.40gb | Legal starts: 0 and 4

GPU A — fragmented

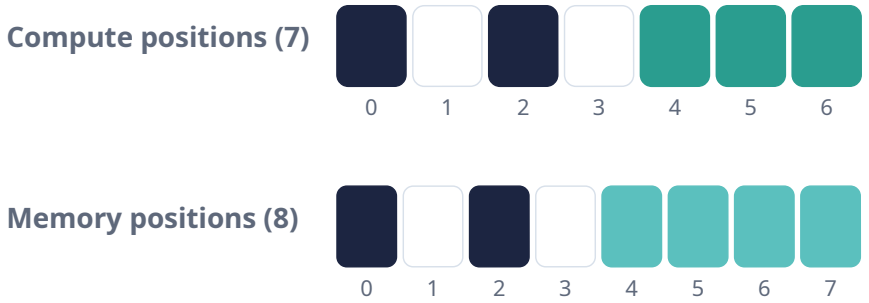


START 0 BLOCKED · START 4 BLOCKED

Aggregate free slices are sufficient, but no legal pair fits.

Blue = occupied | Teal/cyan = candidate footprint

GPU B — legal placement exists



LEGAL START 4 AVAILABLE

Both complete footprints are clear.

Blue = occupied | Teal/cyan = candidate footprint

Illustrative schematic, not measured output. Existing allocations do not migrate in this model.

[1, Figure 3 and Section V-A]

Research Question and Evaluation Objectives

CORE QUESTION

Can bounded lookahead improve future job acceptance enough to justify its extra decision latency?

MODEL

Separate compute, memory, legal starts and job lifetimes.

CONFORMANCE

Audit score semantics against the paper's numeric example.

COMPARISON

Keep paper-path baselines separate from global-feasible alternatives.

EVIDENCE

Use paired traces; report acceptance, latency and uncertainty.

Hypothesis to test — not a claim

Selective lookahead may help when greedy MFI candidates are near-tied, while using less computation than fixed search.

Current scope: acceptance as a primary proxy; fairness, SLOs, energy and priorities are not optimized.

Project research protocol; [1]

Published Foundation: Minimum Fragmentation Increment (MFI)

PUBLISHED METHOD

Zambianco, Fasol & Doriguzzi-Corin

“An Online Fragmentation-Aware GPU Scheduler for Multi-Tenant MIG-based Clouds”

arXiv:2511.18906 (2025)

- Counts blocked legal profile placements.
- Weights each profile by its memory-slice count.
- Chooses a feasible placement minimizing ΔF.

Paper study: 100 simulated A100 GPUs, 4 profile distributions, 500 runs per distribution.

MFI DECISION LOOP

1. **Enumerate** — feasible GPU/start pairs
2. **Dry-run** — hypothetical placement
3. **Measure** — ΔF = Fafter − Fbefore
4. **Select** — argmin ΔF

Published MFI ≠ this project’s lookahead extension

[1]

System Model and Simulator Architecture

LOOKAHEAD CONTEXT — past profile counts + completed-duration history only



Arrival event

Call policy → accept and allocate, or reject. Each decision uses only the current state and permitted history.

Completion event

Release occupied positions; record completed duration for future history-based estimates.

No true future trace or true active-job completion times are revealed to the policy.

Repository: src/mig_scheduler/simulator.py, context.py, state.py

EXPERIMENTAL INPUTS

Synthetic Workload Distributions

Each arriving job requests one of six profiles. Four categorical distributions create different placement pressures.

Profile	Uniform	Skew-small	Skew-big	Bimodal
7g.80gb	16.7%	5%	30%	30%
4g.40gb	16.7%	10%	25%	15%
3g.40gb	16.7%	10%	20%	5%
2g.20gb	16.7%	20%	10%	5%
1g.20gb	16.7%	25%	10%	15%
1g.10gb	16.7%	30%	5%	30%

Uniform
All profiles equally likely.

Skew-small
Many small requests; can block larger profiles.

Skew-big
Large requests have fewer placement options.

Bimodal
Large and small requests dominate together.

[1, Table II; src/mig_scheduler/workloads.py]

COMPARISON SETUP

Policies Compared: From Simple Placement to MFI

Policy family	Decision logic	Role in this project
First Fit (FF)	First available GPU/start under its order	Simple placement baseline
Round Robin (RR)	Rotate GPU choice to spread requests	Load-balancing baseline
Paper-path BF-BI / WF-BI	Choose one GPU by residual capacity, then use Best Index	MIG-aware paper baselines
MFI all-starts	Minimize ΔF under the all-legal-starts score	Prior score interpretation
MFI free-anchor	Minimize ΔF under anchor-filtered score	Figure-3-conformance candidate
Global-feasible BF/WF	Rank every feasible GPU/start pair globally	Separate alternative heuristics; not paper-path

Score mode and baseline-selection semantics are explicit experimental factors—not hidden implementation details.

Repository: src/mig_scheduler/paper_baselines.py, mfi_score_variants.py; [1]

ALGORITHM

How MFI Selects a Placement

Fragmentation score

$$F(g) = \Sigma \; m_p \times Nblocked_p(g)$$

Sum over profiles p with enough aggregate free compute AND memory.

---

m_p profile memory-slice count

$Nblocked_p(g)$ blocked legal starts counted by selected score mode

$\Delta F(g,i) = F(\text{after placement}) - F(\text{before})$

MFI selects the feasible pair (g*, i*) with the smallest ΔF.

1. **Check aggregate resources**
   free compute ≥ profile compute AND free memory ≥ profile memory
2. **Check legal footprint**
   full compute and memory ranges at start i do not overlap
3. **Evaluate a hypothetical state**
   recompute score under the selected interpretation
4. **Take argmin ΔF**
   tie-break deterministically by GPU ID, then start index

Score is a flexibility heuristic, not a percentage of wasted GPU capacity.

[1, Algorithm 1 and Algorithm 2; src/mig_scheduler/fragmentation.py]

M12 Audit: Reconstructing the Paper's Figure 3 Scores

Reported in the paper: F(GPU 1) = 8 and F(GPU 2) = 16

Reconstructed state	Compute occupancy	Paper score	free_anchor_starts	all_legal_starts
GPU 1: 1g.10gb @ 2, @ 3	2 / 7 = 28.6% ≈ 30%	**8**	**8**	14
GPU 2: 2g.20gb @ 2; 1g.10gb @ 5	3 / 7 = 42.9% ≈ 43%	**16**	**16**	23

all_legal_starts

Counts every blocked legal start when aggregate compute and memory are sufficient.

free_anchor_starts

Skips starts whose anchor index is already occupied in either resource dimension, then checks the full footprint.

Conformance clue, not proof: anchor filtering is not explicit in the paper pseudocode; its executable source was unavailable.

[1, Figure 3 and Algorithm 1; M12_FIGURE3_CONFORMANCE.md]

M12 Audit: Correct Baseline Semantics

Paper-path BF-BI / WF-BI: choose a GPU first, then try Best Index starts on that GPU.

1 Rank GPUs

Only GPUs with sufficient aggregate compute AND memory enter the ranking.

2 Select one GPU

BF minimizes the documented residual score; WF maximizes it.

3 Try Best Index

Inspect legal starts on the selected GPU in profile-specific order.

4 Accept or reject

If no legal start exists there, reject—even if another GPU could fit.

Project residual score assumption

$$R(g,p) = (Cfree - C_p)/7 + (Mfree - M_p)/8$$

BF minimizes R; WF maximizes R.

Why this audit matters

Global-feasible BF/WF search all feasible GPU/start pairs and are kept as separate alternative heuristics—not mislabeled as paper-path baselines.

M12_BASELINE_SEMANTICS.md; [1]

EXTENSION

Fixed Lookahead: Evaluate Future Admission, Not Only Immediate Fragmentation

01

Enumerate

All feasible current placements a

02

Roll out

Same sampled future job plans per candidate

03

Count

Future jobs admitted within horizon H

04

Choose

Candidate with highest mean future acceptance

Estimated future admission value

$$\hat{V}H(a) = (1 / R) \ \Sigma_r \ Y_r(a)$$

$Y_r(a)$ = number of later jobs admitted in rollout r; the current job is excluded.

M13 SETTINGS

H = 2

3 rollouts per feasible current placement candidate

Tie-break: lower immediate ΔF → GPU ID → start index.

Future workload and residual lifetimes are sampled/predicted from available history; true future ground truth is not exposed.

LOOKAHEAD_METHOD.md; src/mig_scheduler/schedulers.py

EXTENSION

Adaptive Lookahead: Spend Search Only When It Might Matter

Profile estimate from past arrivals

$$\hat{q}_p = (n_p + \eta) / (N + K\eta)$$

Compare older vs. newer halves of recent profile history:

$$TV(\hat{q}old, \hat{q}new) = \tfrac{1}{2} \Sigma_p \, |\hat{q}old,_p - \hat{q}new,_p|$$

$$Rt = \max(0, 1 - TV / \theta TV)$$

Stability score and Student-t stopping interval are heuristics—not calibrated probabilities or formal error guarantees.

Decision gates

Insufficient history? →	Use greedy MFI
Recent mix unstable? →	Use greedy MFI
Top ΔF candidates not close? →	Use greedy MFI
Otherwise →	Shortlist ≤5 and run rollouts

M13: H = 2 · batch size 2 · 2–4 rollouts · ≤5 candidates

ADAPTIVE_LOOKAHEAD_METHOD.md; M13_SCORE_MODE_DESIGN.md

Implementation: Explicit Score Mode Through the Entire Decision Path

One contract, end to end

score_mode	all_legal_starts | free_anchor_starts
current ranking	ΔF for feasible current placements
future rollouts	same score mode for later placements
tie-breaking	(ΔF, GPU ID, start index)
adaptive fallback	greedy MFI uses declared mode too

M13 fix: consistent score_mode end-to-end.

Repository modules

profiles.py / state.py	Profile geometry, legal starts, occupancy
fragmentation.py / mfi_score_variants.py	Score functions + greedy MFI
schedulers.py / context.py / simulator.py	Fixed/adaptive search + online boundary
workloads.py / scripts	Trace generation, paired runs, CSVs, manifests

Deterministic candidate ordering makes paired comparisons reproducible.

src/mig_scheduler/mfi_score_variants.py; src/mig_scheduler/schedulers.py; CHANGELOG_M13.md

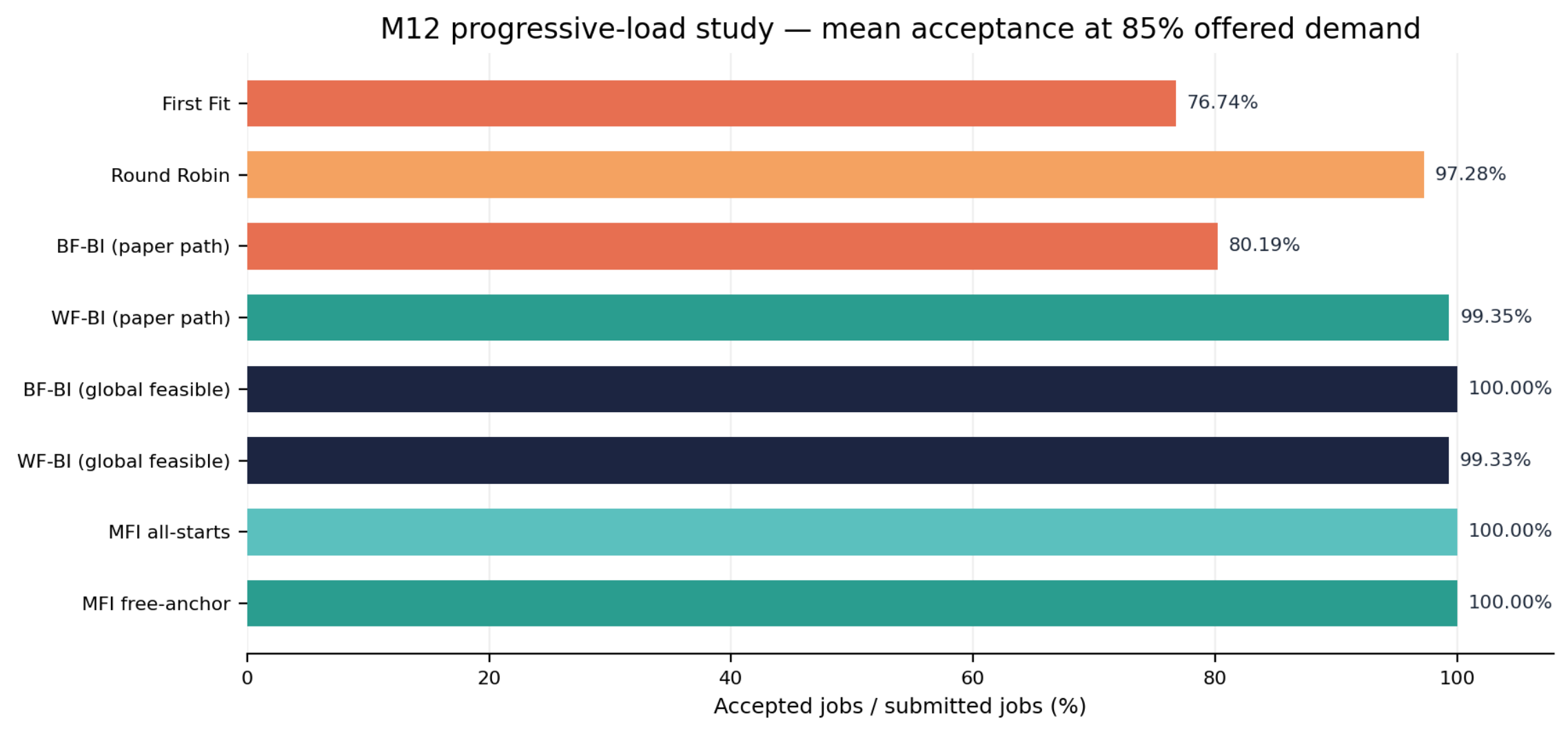
How the Research Evolved: Milestones 1–13

1	M1–M3	**Core model**	Simulator, profile/placement state, greedy MFI and fixed-lookahead prototype.
2	M4–M6	**Algorithm checks**	Adaptive budget hypothesis, tiny-trace offline oracle, matched budget frontier.
3	M7–M10	**Protocol audits**	Horizon, workload dynamics, arrival spacing and load-basis sensitivity.
4	M11–M12	**Conformance**	Resource gates, BF/WF semantics, Figure 3 score and baseline audits.
5	M13	**Controlled test**	Score-aware fixed/adaptive lookahead under paired progressive and burst loads.

The repository includes source, tests, manifests, per-job decisions, aggregate CSVs, plots and decision memos.

Repository changelogs M6–M13

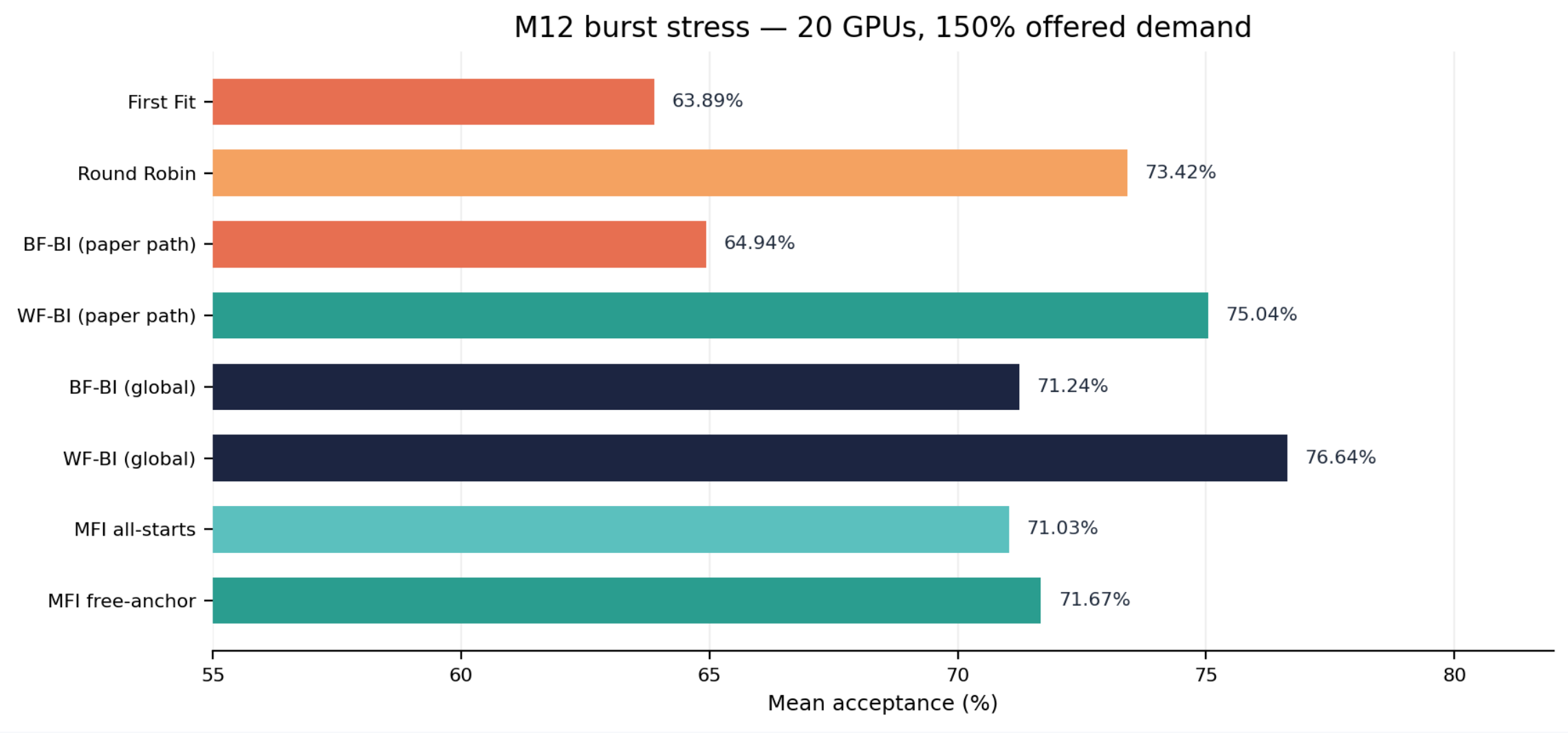
M12 Result 1: Progressive Load on 100 Simulated GPUs



100 GPUs · 4 distributions · 5 reps/distribution · 8 policies (160 runs); mean acceptance at 85% requested-demand checkpoint.

results/m12_figure3_baseline_study_100gpu_5rep/M12_EXPERIMENT_REPORT.md; [1]

M12 Result 2: Burst / High-Demand Stress Test



20 GPUs · 150% offered demand · gap 0.0 · 5 reps/distribution · 160 runs. Free-anchor − all-starts = +0.64 pp; paired interval −0.80 to +1.86 pp includes zero.

M13 Experiment: Paired Evaluation of Score-Aware Lookahead

Cluster
4 A100-profile GPUs (abstract model)

Workloads
Uniform, skew-small, skew-big, bimodal

Replications
3 per distribution × 2 regimes = 24 shared traces

Policies
10 variants × 24 traces = 240 simulations

Two arrival regimes

Progressive	100% offered-demand target · inter-arrival gap 0.25
Burst / stress	150% offered demand · gap 0.0 (all arrivals before completion)

Search settings

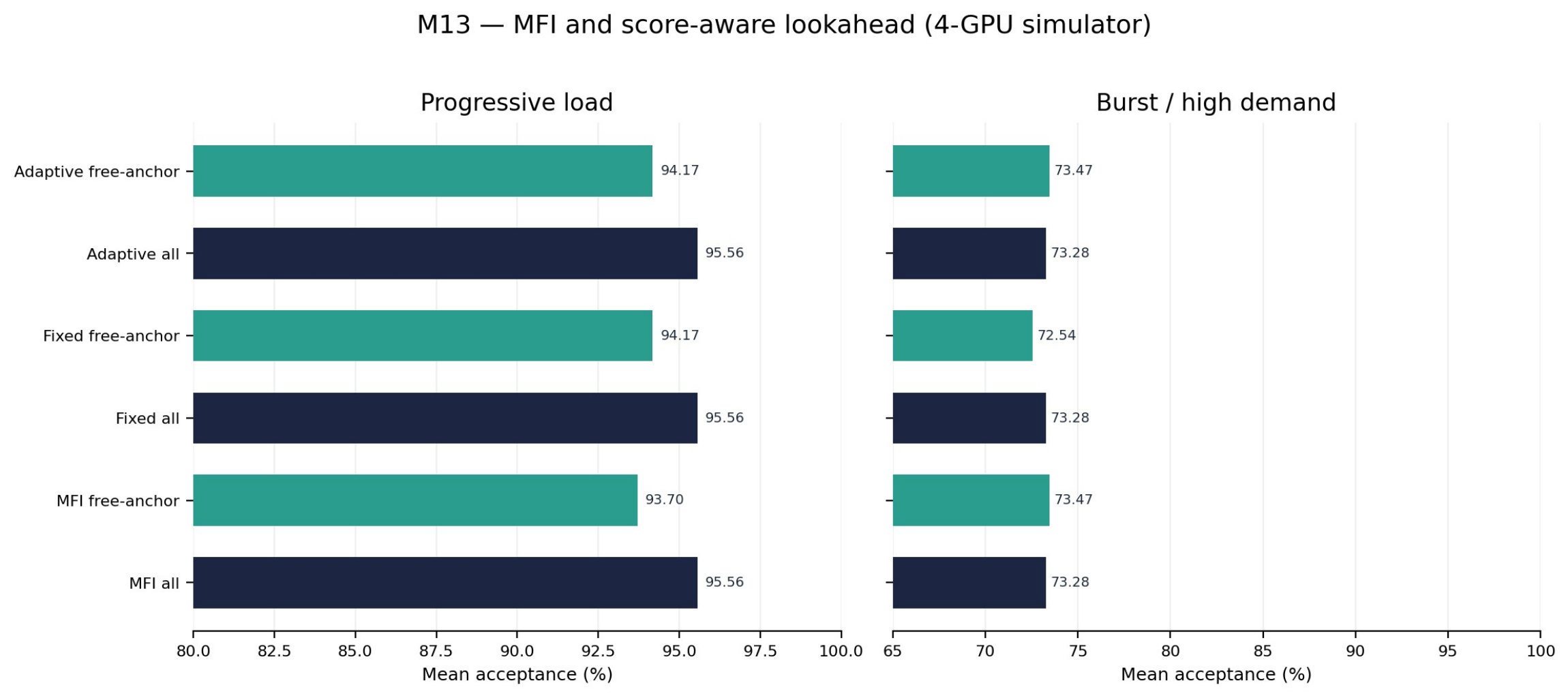
Fixed: H = 2; 3 rollouts per feasible current candidate.

Adaptive: H = 2; 2–4 rollouts, batch size 2, at most 5 candidates.

Matched traces isolate policy differences; sample size remains exploratory.

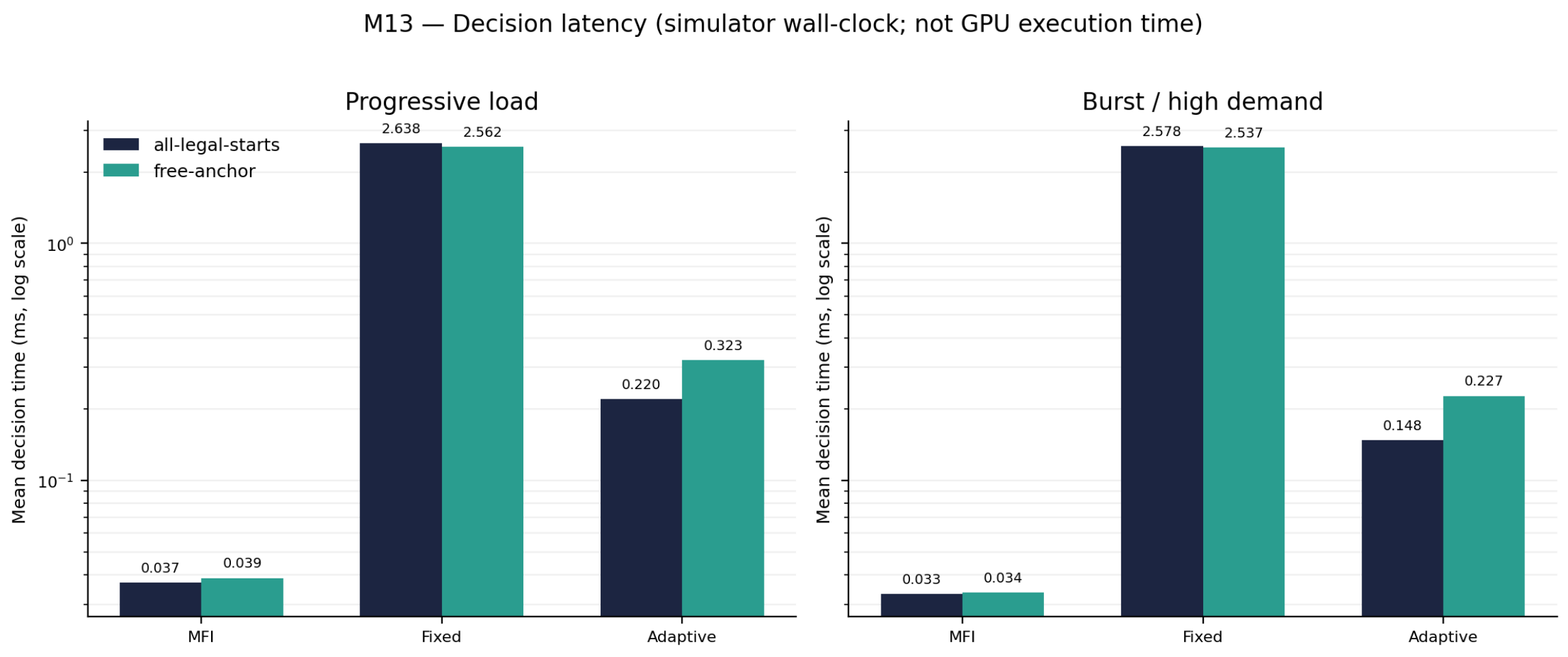
M13_EXPERIMENT_REPORT.md; m13_run_manifest.json

M13 Results: Acceptance Does Not Show a Consistent Lookahead Win



Only 12 paired traces per regime: differences are exploratory, not evidence of a robust gain.

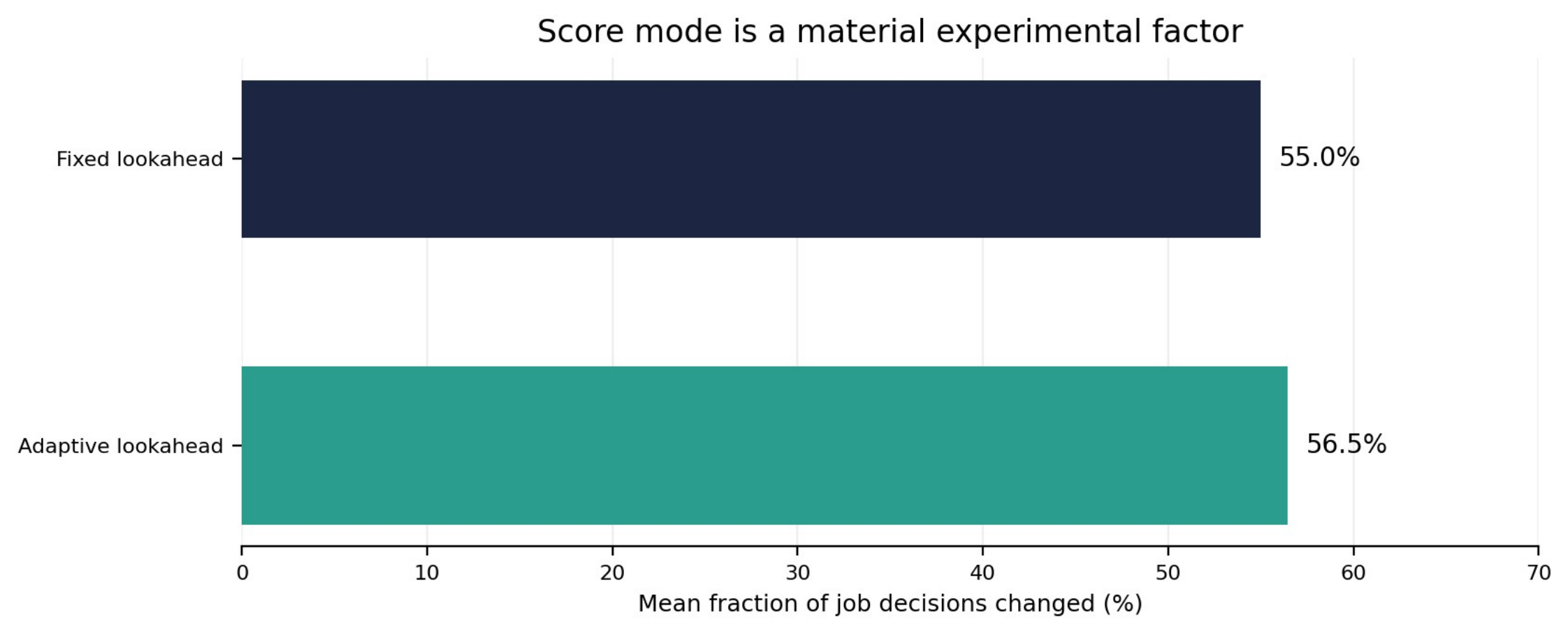
M13 Results: Decision Latency and Search Frequency



Simulator wall-clock time (ms), logarithmic axis; machine/process dependent—not GPU kernel time.

M13_EXPERIMENT_REPORT.md; m13_policy_summary.csv

M13 Audit: Score Semantics Change Many Decisions



Fixed lookahead: 55.0% of per-job decisions changed

Adaptive lookahead: 56.5% changed

A changed tuple = (status, GPU ID, start index); it may not change job acceptance.

m13_score_mode_decision_sensitivity.csv; M12_FIGURE3_CONFORMANCE.md

RELIABILITY

Validation and Reproducibility

TESTS

- 107 automated tests passed
- python -m pytest -q
- Regression tests cover score-aware lookahead and placement constraints.

BASELINE CHECKS

- Profile-model validation
- Memory-overlap regression
- MFI argmin consistency over 250 deterministic randomized states.

EXPERIMENT ARTIFACTS

- Raw runs + per-job decisions
- Aggregate summaries + paired deltas
- Bootstrap intervals + JSON run manifest.

What tests do not establish **Implementation consistency is not proof of exact performance replication or live NVIDIA driver behavior.**

Python 3.11+ recommended · python -m pip install -e ".[test,analysis]"

M13_RUN_ME_FIRST.md; scripts/validate_baseline.py

Limitations, Threats to Validity, and Next Experiments

Limit	Implication	Concrete next action
Simulation fidelity	Abstract profile/placement model; not driver-validated MIG emulation or hardware benchmark.	Validate model assumptions against supported configurations before hardware claims.
Synthetic workloads	No real tenant arrival/profile/duration dataset.	Use trace-driven or calibrated workload distributions.
Small M13 sample	3 repetitions per distribution/regime; 12 paired traces per regime.	Expand to ≥20–30 held-out seeds after calibration.
Score ambiguity	Free-anchor matches two Figure 3 values; original executable behavior remains unconfirmed.	Keep both score modes in all sensitivity studies.
Policy heuristics	BF/WF residual scalarization and adaptive stop thresholds are assumptions.	Preregister alternatives and calibrate only on separate tuning seeds.
Objective scope	Acceptance only; fairness, SLOs, energy, migration and priorities are not optimized.	Add operational constraints only after admission/latency behavior is stable.

Next experiment: preregister a larger paired study with held-out seeds; report acceptance, latency, fallback rate and variability together.

M13_FINDINGS_AND_NEXT_DECISION.md; ADAPTIVE_LOOKAHEAD_METHOD.md

TAKEAWAYS

Conclusion: What We Know Today

01	**MFI is a useful baseline**	It explicitly minimizes immediate increase in profile-specific fragmentation.
02	**Score semantics matter**	free_anchor_starts matches reported Figure 3 scores 8 and 16 on reconstructed states.
03	**Baseline labels need precision**	Paper-path GPU-first BF/WF are distinct from global-feasible alternatives.
04	**M13 made the comparison consistent**	Score mode now reaches current placement, rollout placement, tie-breaking and fallback.
05	**No stable lookahead win established**	Fixed search adds substantial decision latency in the current small sample.

Research decision **Do not claim superiority yet. Expand the paired held-out study and keep both score modes explicit.**

SOURCES

References

[1] **Marco Zambianco, Lorenzo Fasol, and Roberto Doriguzzi-Corin.**

"An Online Fragmentation-Aware GPU Scheduler for Multi-Tenant MIG-based Clouds." arXiv:2511.18906, 2025.

https://arxiv.org/abs/2511.18906

[2] **NVIDIA.**

"Multi-Instance GPU User Guide" — concepts, supported MIG profiles and MIG device names.

https://docs.nvidia.com/datacenter/tesla/mig-user-guide/

[3] **Bekir Turkkan, Pavankumar Murali, Pavithra Harsha, Rohan Arora, Gerard Vanloo, and Chandra Narayanaswami.**

"Optimal Workload Placement on Multi-Instance GPUs." arXiv:2409.06646, 2024.

https://arxiv.org/abs/2409.06646

Project artifacts

M12_FIGURE3_CONFORMANCE.md · M12_BASELINE_SEMANTICS.md · M13_SCORE_MODE_DESIGN.md · M13_EXPERIMENT_REPORT.md · M13_FINDINGS_AND_NEXT_DECISION.md

External sources [1]–[3]; project files listed above.

Appendix A — MFI Pseudocode and Feasibility Checks

PRESENTATION PSEUDOCODE

```
INPUT state S; arriving profile p; score_mode
CANDIDATES ← ∅
for each GPU g:
  if free_compute(g) < compute(p): continue
  if free_memory(g) < memory(p): continue
  F_before ← F(g, score_mode)
  for each legal start i:
    if compute OR memory footprint overlaps: continue
    S′ ← hypothetical placement (g, i)
    ΔF ← F(S′, score_mode) − F_before
    append (ΔF, GPU_ID, i) to CANDIDATES
if CANDIDATES = ∅: reject
else: allocate argmin (ΔF, GPU_ID, start index)
```

Two independent gates

- Aggregate free compute must be enough.
- Aggregate free memory must be enough.

REJECT if no legal pair

Free totals alone do not establish a legal start.

Complexity note

The paper describes main decision cost as scaling with GPU count; profile/start enumeration is bounded by the fixed hardware profile table.

Derived presentation pseudocode, not a byte-for-byte quotation. Source: [1, Algorithm 2] + project implementation.

[1, Algorithm 2]; src/mig_scheduler/mfi_score_variants.py

Appendix B — Reproduce the M13 Experiment

FROM THE REPOSITORY ROOT · PYTHON 3.11+ RECOMMENDED

```
python -m venv .venv
# Windows PowerShell: .venv\Scripts\Activate.ps1
# Linux/macOS: source .venv/bin/activate
python -m pip install -e ".[test,analysis]"
python -m pytest -q
python scripts/validate_baseline.py
python scripts/run_m13_score_aware_lookahead.py --gpus 4 --repetitions 3 --seed 20261026 --out results/m13_score_aware_lookahead_4gpu_3rep
```

KEY OUTPUTS

m13_raw_runs.csv	Run-level outcomes
m13_per_job_decisions.csv	Job choices + search diagnostics
m13_policy_summary.csv	Acceptance, utilization and runtime
m13_paired_acceptance_vs_free_anchor_mfi.csv	Paired deltas + bootstrap intervals
m13_score_mode_decision_sensitivity.csv	Changed-decision fraction by policy
m13_run_manifest.json	Exact configuration and seed

Matched traces are the key control; wall-clock timing may vary by machine/process.